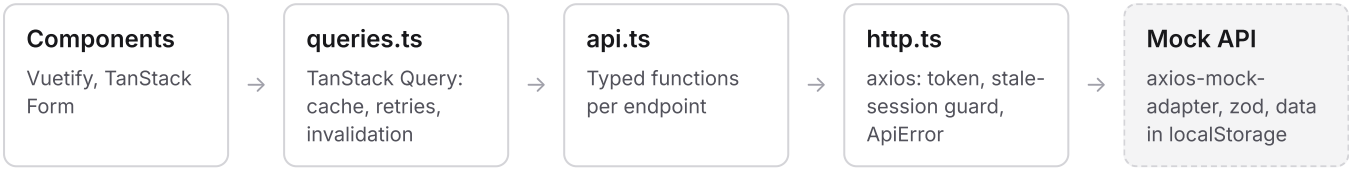
OrderDesk

The brief fixed Vue 3, the Composition API, TypeScript and axios-mock-adapter. Every other library replaces code I would otherwise write and test myself; anything the brief didn't need was left out.

How a request travels



Session store (Pinia)
token · user · permissions from `GET /me`

Read by route guards, interceptors and components. Components never touch axios, and every failure reaches them as one typed `ApiError`. Set `VITE_API_BASE_URL` and the mock is never loaded.

The stack

LAYER	CHOICE	WHY	INSTEAD OF
SCAFFOLD	**create-vue** Vite 8.3 · TS 6.0	The official Vue starter, so the layout is familiar to any Vue developer. TypeScript stays on 6 because vue-tsc still needs its JavaScript compiler API.	Nuxt: no SSR or SEO need in an internal tool
UI	**Vuetify** 4.3	Accessible fields, dialogs with focus trap and scroll lock, tables, snackbars, skeletons and progress bars. Its theme and global defaults hold every shared style in one file.	Tailwind and a headless kit: every field, table and toast built by hand
FORMS	**TanStack Form** 1.33	Typed nested fields for line items, validation on submit and then as you type, and a slot for each field's 422 error.	VeeValidate: equally capable; TanStack matches the Query model
SERVER STATE	**TanStack Query** Vue 5.104	One cache entry per filter, so a slow older search can't overwrite a newer one. Also cancellation, refetch after writes, and one `clear()` on sign-out.	Loading flags and caching written by hand in Pinia
SESSION	**Pinia** 4.1 · one store	Token, user and permissions. Router guards and interceptors read it as well as components, and each test gets a fresh one.	A module-level reactive object: shared between tests, no devtools
HTTP	**axios** 1.20	The brief's mock adapter plugs into it. Interceptors attach the token, drop responses from an ended session and turn every failure into one typed `ApiError`.	fetch: the brief's mock adapter needs axios
VALIDATION	**zod** 4.6	One schema for the form and the mock, so the API's 422 paths (`items.1.quantity`) are the form's field names.	Separate client and server rules that drift apart
PHONE NUMBERS	**libphonenumber-js** 1.13	Knows which Iraqi numbers are mobiles, in any notation, and stores one format. Loaded only with the order panels.	A hand-written regex
TESTS	**Vitest** 4.1 · Testing Library	UI tests render the whole app, wired as in production, against the mock with no delay, and act like a user. Contract tests call the mock API directly.	Shallow component tests that miss the wiring

Also used: path-to-regexp for the mock's route patterns (`/orders/:id`), VueUse for event listeners that clean up after themselves, and MDI icons imported one by one as SVG paths.

Decisions that shape the code

01 Permissions, never roles

Code asks `session.can('orders.cancel')`. Roles exist only on the server, so a new role needs no client change. With no permission, an action is **hidden**: it never applies to this user. Allowed but blocked by the order's state, it is **disabled with the reason** on screen.

02 The server has the final say

The mock re-checks every rule in a fixed order: 401 → 403 → 404 → 409 → 422. Responses are built field by field from an allow-list, so a new field never reaches Support by accident. A 403 is about the user, so `/me` is re-read; a 409 is about the order, so the order is refetched.

03 A session ends one way

Sign-out and a 401 share `endSession()`: forget the token, go to sign-in, clear the query cache and pending saves, reset the store, revoke the token. Responses sent with an old token are discarded, so nothing reaches the next user. A 403 never signs anyone out.

04 The URL is the state

Filters and the open panel live in the URL (`/orders/12?status=pending`), so refresh, Back and shared links work. Search waits for a 300 ms pause, and the list always matches the search box.

05 One form, one schema

Create and edit share one form and the zod schema the mock validates with. 422 errors appear under their fields and stay with their line item when another is removed. Edit works on a copy, so a background refresh never overwrites typing.

06 Failure is part of the demo

The Mock API panel makes the next order request fail with a 500, network error, 401, 403 or 422, then switches itself off, so a reviewer sees the error and then the recovery. It works over an open order panel too. Loads retry once, saves never.

07 Loaders sit on what's loading

Skeletons have the shape of the table, list, panel and form they stand in for, so nothing moves when data arrives. A search or filter puts a bar on the results, not in the search box; a page still downloading puts one under the app bar.

08 One look in dev and production

Vuetify ranks its CSS in cascade layers, ordered by first appearance, and the build links component CSS before the file that declares them. `index.html` declares the order first; without it, built buttons drew 16 px text where dev showed 14 px.

Trade-offs I accepted

Token in `localStorage`	The mock runs in the browser. In production the token becomes an httpOnly cookie, a change confined to `http.ts` and the auth feature.
409, not 403, for final orders	Changing a delivered or cancelled order fails even for Admin. The block comes from the order's state, not the user's rights.
One rule beyond the brief	Creating or editing also needs `orders.view_amount`: orders carry prices, and a role that can't see them shouldn't overwrite them.
No pagination	Fifty orders fit in one response. Server-side pagination and sorting are the first next step.
~34 KB gzipped for phone numbers	libphonenumber-js loads only with the order panels. Iraq-only metadata would cut it to a few KB.

Deliberately not added: a date library (`Intl` formats IQD and dates), an i18n library, stores for UI state (the URL holds it), and CI with Playwright end-to-end tests, which the README lists as next steps.

More detail: **README.md** (setup, test users, permission model, connecting a real backend) and **docs/project-report.pdf** (each requirement in the brief, with the passing tests that prove it).